Machine Learning: The No-Nonsense (But Fun) Guide

Day 12: AdaBoost – The "Focus on Your Mistakes" Algorithm 🎯

---

1. Story & Intuition

Imagine you're learning to play basketball. You take 100 shots.

A coach analyzes your performance:

- ✅ 70 shots: You made them. Move on.
- ❌ 20 shots: You missed left. Practice these more.
- ❌ 10 shots: You missed right. Practice these too.

In the next practice session, you spend more time practicing the shots you previously missed.

You continue repeating this process until your weaknesses become strengths.

This is exactly how AdaBoost (Adaptive Boosting) works.

Instead of treating every training example equally, AdaBoost pays more attention to the samples that previous models predicted incorrectly.

> **"Random Forest: Everyone votes at once. AdaBoost: Learn from your mistakes, one step at a time."**

---

2. What is AdaBoost?

AdaBoost (Adaptive Boosting) is a boosting algorithm that combines multiple weak learners to create one strong learner.

Unlike Random Forest, where trees are trained independently, AdaBoost trains models one after another, and each new model focuses on correcting the mistakes made by the previous one.

Random Forest vs AdaBoost

Random Forest	AdaBoost
Trees grow independently	**Trees grow sequentially**
Equal voting	**Better trees receive more vote**
Reduces variance	**Reduces bias**
Parallel training	**Sequential training**
Usually deep trees	**Usually shallow trees (Decision Stumps)**

Core Idea

- Weak Learner: Slightly better than random guessing.
- Strong Learner: Combination of many weak learners producing high accuracy.

The weak learner used in AdaBoost is usually a Decision Tree with depth = 1, known as a Decision Stump.

---

3. Working of AdaBoost

Step 1 – Assign Equal Weights

Initially, every training sample is equally important.

Samples:
[A, B, C, D, E, F, G, H]

Weights:
[1/8, 1/8, 1/8, 1/8, 1/8, 1/8, 1/8, 1/8]

Each sample has the same probability of being selected.

Step 2 – Train the First Weak Learner

A simple Decision Stump is trained.

Example:

Tree 1

Question:
Is Height > 5 ft?

Correct:
A, B, C

Wrong:
D, E, F

Calculate the weighted error.

Error =
Sum of weights of all incorrectly classified samples

Step 3 – Calculate Alpha (α)

Alpha represents the importance (weight) of the weak learner.

Formula:

$\alpha = ½ \times \ln((1 - \text{Error}) / \text{Error})$

Error	Alpha Interpretation
0.1	Large positive value → Strong learner
0.5	Zero → No contribution
0.9	Negative value → Prediction should be inverted

Lower error results in higher Alpha.

Step 4 – Update Sample Weights

AdaBoost now changes the importance of every training sample.

- Increase weights for incorrectly classified samples.
- Decrease weights for correctly classified samples.

Weight update equations:

Wrong Prediction

New Weight = Old Weight × $e^{(\alpha)}$

Correct Prediction

New Weight = Old Weight × $e^{(-\alpha)}$

After updating, all weights are normalized so that their total equals 1.

Illustration

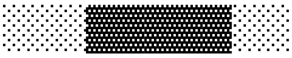
Before Training

A B C D E F G H

(All samples have equal importance)

After First Weak Learner

A B C D E F G H



Wrong samples (D,E,F)
receive higher weights.

These samples receive greater attention during the next iteration.

Step 5 – Repeat

Train another weak learner using the updated sample weights.

Each subsequent learner focuses more on the previously misclassified samples.

Example:

- Tree 2 focuses on D, E, F.
- Tree 3 focuses on remaining mistakes.
- Continue until the specified number of estimators is reached.

Step 6 – Final Prediction

Instead of majority voting, AdaBoost combines predictions using weighted voting.

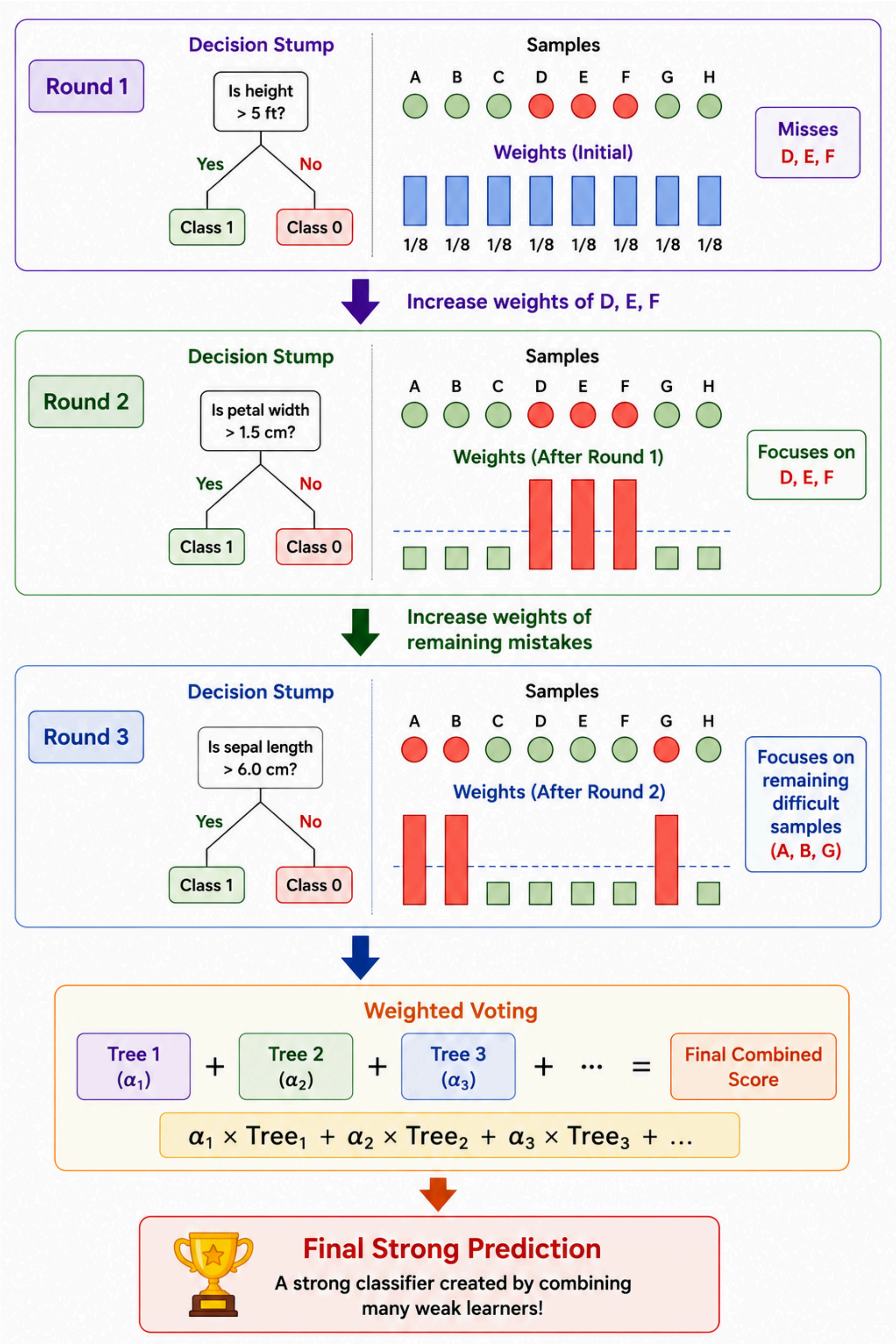
Final Prediction

$= \alpha_1(\text{Tree}_1)$
$+ \alpha_2(\text{Tree}_2)$
$+ \alpha_3(\text{Tree}_3)$
$+ \ldots$

Weak learners with lower errors receive larger Alpha values and therefore have more influence on the final prediction.

---

4. Visual Representation –



---

5. Important Hyperparameters

Hyperparameter	Description	Recommended Value
estimator	Weak learner	Decision Tree (depth=1)
n_estimators	Number of boosting rounds	50–500
learning_rate	Controls contribution of each learner	0.1–1.0
algorithm	Boosting algorithm	SAMME / SAMME.R

Trade-off

Lower Learning Rate

↓

Needs More Estimators

Higher Learning Rate

↓

Needs Fewer Estimators

A common combination is:

learning_rate = 0.1

n_estimators = 500

---

6. AdaBoost vs Random Forest

Feature	AdaBoost	Random Forest
Training	Sequential	Parallel

Tree Type	Shallow Stumps	Deep Trees
Tree Importance	Weighted	Equal
Main Goal	Reduce Bias	Reduce Variance
Training Speed	Slower	Faster
Noise Sensitivity	High	Lower

---

7. When to Use AdaBoost

Suitable For

- **Simple classification problems**
- **Boosting weak learners**
- **Small to medium datasets**
- **Understanding boosting concepts**

Avoid When

- **Very noisy datasets**
- **Large numbers of outliers**
- **High-dimensional sparse datasets**
- **Need highly parallel computation**

---

8. Common Problems

Problem	Cause
Overfitting	**Focuses too much on noisy samples**
Outlier Sensitivity	**Outliers receive very high weights**

Slow Training	**Sequential learning**
Class Imbalance	**Majority class dominates**

---

9. Real-World Applications

- Face Detection
- Text Classification
- Medical Diagnosis
- Spam Detection
- Customer Risk Prediction

---

10. Interview Questions

1. What is AdaBoost?
2. What is a Weak Learner?
3. Why are Decision Stumps commonly used?
4. How are sample weights updated?
5. What is Alpha (α)?
6. Why is AdaBoost called Adaptive Boosting?
7. Difference between AdaBoost and Random Forest?
8. What is the role of Learning Rate?
9. What happens when error becomes greater than 0.5?
10. What are the limitations of AdaBoost?

Happy Learning !!!